

---

**Cours n° 1**

---

---

**STRUCTURES SÉQUENTIELLES**

---

**I BOUCLES CONDITIONNELLES**

Une boucle conditionnelle répète une suite d'instructions tant qu'une condition est vérifiée :

```
Tant que <condition>
    faire <instruction>
Fintantque
```

Exemple :

```
Tant que je ne connais pas mes questions de cours
    faire apprendre les questions de cours
Fintantque
```

La syntaxe en python est simple :

```
1 while condition :
2     instructions
```

Tout le code indenté après les : est dans la boucle. Pas besoin de dire "fin tant que", il suffit de désindenter.

EXEMPLE 1. Parcourir pas à pas la boucle suivante

```
1 a = 159
2 while a != 0 :
3     print(a)
4     a = a // 10
```

**MÉTHODE**

Quand penser à utiliser une boucle while? Quand la suite du processus dépend d'une condition dont on ne connaît pas a priori quand elle sera vraie :

- La réussite d'un test aléatoire
- Dépasser un certain seuil
- Plein d'autres cas!

**II BOUCLES INCONDITIONNELLES**

Une boucle inconditionnelle répète une suite d'instruction un nombre de fois donné.

```
Pour k dans <ensemble>
    faire <instruction>
Fintantque
```

**MÉTHODE**

Quand penser à utiliser une boucle for? Quand le processus en lui même se fait avec un nombre d'étapes fixe.

- Analyser les éléments d'un tableau
- Calculer un terme d'une suite
- Plein d'autres cas!

## 1 PARCOURS DE SÉQUENCE

En python on dépasse un peu la notion de lycée : une boucle inconditionnelle parcourt une séquence.

### DÉFINITION 1

Une **séquence** est une variable informatique que l'on peut parcourir "pas à pas" de manière ordonnée. Les cas les plus courants sont les séquences d'entiers, les listes et les chaînes de caractères.

```
1 for elt in sequence :
2     instructions
```

La syntaxe est usuelle : ce qui est indenté est dans la boucle.

EXEMPLE 2. Parcourir pas à pas la boucle suivante

```
1 for k in [5,3,1,5,4] :
2     n = 0
3     if k == 5 :
4         n += 1
```

## 2 RANGE

### DÉFINITION 2

La commande range permet de faire une séquence d'entiers. Soit trois formes possibles :

- `range(n)` va de 0 à  $n-1$  de 1 en 1 : 0, 1, 2, ...,  $n-1$
- `range(a, b)` va de  $a$  à  $b-1$  de 1 en 1 :  $a, a+1, a+2, \dots, b-2, b-1$
- `range(a, b, k)` va de  $a$  à maximum  $b-1$  de  $k$  en  $k$  :  $a, a+k, a+2k, \dots, a+nk$  avec  $a+nk \leq b-1 < a+(n+1)k$

EXEMPLE 3. Parcourir pas à pas le code suivant.

```
1 n = 1
2 for i in range(2,5):
3     n = n*i
```

## 3 VALEURS OU POSITIONS?

Un tableau peut se parcourir en valeurs ou en position, ces deux cas sont à retenir :

```
1 for val in T : # valeur par valeur
2
3 for k in range(len(T)): # position par position
```

EXEMPLE 4. Analyser les deux cas avec `T = [5, 3, 1, 9, 2]`.

Lequel est le mieux? Comme toujours, cela dépend de ce que l'on veut faire.

## III FONCTIONS ET BOUCLES

On peut (et on fera quasiment que ça) mettre des boucles dans des fonctions.

Une règle importante : l'instruction `return` termine la fonction, et donc la boucle, même si il "reste" des éléments à parcourir.

EXEMPLE 5. Avec l'argument `[3, 1, -2, 4, -6, 3]` la fonction s'arrête au 3e passage en boucle.

```
1 def toutpositif(T):
2     for val in (T):
3         if val < 0 :
4             return False
5     return True
```

Ce principe est fondamental pour beaucoup, beaucoup de parties du programme.

## IV COMPTEURS

Un **compteur** permet de compter (d'où le nom...). Par exemple le nombre de passage en boucle dans un while, ou le nombre de tests réussit dans un for.

EXEMPLE 6. Cette fonction permet de déterminer le nombre de chiffres d'un entier  $n$  écrit en décimal.

```
1 def taille(n):
2     ctp = 0
3     while n > 0 :
4         ctp = ctp + 1
5         n = n // 10
6     return ctp
```

Un **accumulateur** est comme un compteur, sauf qu'on n'ajoute pas nécessairement 1.

EXEMPLE 7. Un calcul de moyenne.

```
1 def moyenne(L):
2     acc = 0
3     for val in L :
4         acc = acc + val
5     return acc / len(L)
```

## V EXERCICES

EXERCICE 1. Écrire une version qui détermine le nombre de chiffres d'un entier  $n$  dans son écriture décimale.

EXERCICE 2. Écrire une fonction permettant de construire la chaîne de caractères contenant tous les entiers de 0 à  $N - 1$ . Par exemple `chaine(12)` renverra '012345678910112'. Aide : la commande `str(i)` converti l'entier  $i$  en une chaîne de caractère.

EXERCICE 3. Écrire une fonction qui prends en argument une chaîne de caractère et un caractère, et renvoie le nombre de fois ou le caractère est présent dans la chaîne. Par exemple `nombredecar('Jean Robert est sympa', 'e')` renvoie 3.

EXERCICE 4. Écrire une fonction `sommeabs(L)` qui renvoie la somme des valeurs absolues des nombres la liste  $L$ . Par exemple `sommeabs([3, -2, 5])` renvoie 10.

EXERCICE 5. Des valeurs de  $n$  posent-elles un petit problème?

```
1 n = # entier à saisir
2 while n != 1:
3     if n > 4 or n%2==0 :
4         n = n//2
5     else:
6         n = 2*n-3
7     print(n)
```

EXERCICE 6. Dans la bibliothèque `random` l'appel `random.randint(1, 6)` permet d'obtenir de manière aléatoire un entier entre 1 et 6 (bref un d6). Écrire :

- 1) Une fonction qui retourne le nombre de lancers avant d'obtenir un 6.
- 2) Une fonction qui retourne le nombre de lancers avant d'obtenir  $n$  6 en tout.
- 3) Une fonction qui retourne le nombre de lancers avant d'obtenir  $n$  6 de suite.